

基于静态分析和模糊测试的路由器漏洞检测方法

王洪义¹ 沙乐天^{1,2}

¹(南京邮电大学计算机学院、软件学院、网络空间安全学院 南京 210023)

²(江苏省无线传感网高技术研究重点实验室 南京 210023)

(hongyi0228@163.com)

Router Vulnerability Detection Method Based on Static Analysis and Fuzzing

Wang Hongyi¹ and Sha Letian^{1,2}

¹(School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023)

²(Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks, Nanjing 210023)

Abstract Network attacks targeting router devices often have serious consequences. Fuzzing testing is an effective method to detect security vulnerabilities in router devices. However, without sufficient analysis of the firmware of the target device, fuzzy testing is often blind and ineffective. In this paper, we propose a method of using static analysis assisted fuzzy testing to detect vulnerabilities in router devices. Specifically, the results generated by static analysis are used to construct more effective test cases to fuzz the web interface of the router device. Our opinion is that there is a lot of useful information hidden in the router firmware. We use static analysis to extract the possible loopholes in the program code to build test cases and improve the efficiency of fuzzing. We implemented a prototype system and tested it on 46 router firmware from 4 mainstream router vendors, and found 16 vulnerabilities, 4 of which were 0-day vulnerabilities. The results show that our system can detect vulnerabilities that cannot be detected by existing vulnerability detection tools compared to advanced automated vulnerability mining methods.

Key words static analysis; fuzzing; firmware; vulnerability mining; Web interface

摘要 针对路由器设备的网络攻击往往会造成严重后果。模糊测试是检测路由器设备安全漏洞的有效方法。然而,如果没有对目标设备的固件进行足够的分析,模糊测试往往是盲目和无效的。提出一种使用静态分析辅助模糊测试对路由器设备进行漏洞检测的方法。具体来讲,就是通过静态分析生成的结果指导测试用例的变异来对路由器设备的 Web 接口进行模糊测试。路由器固件中隐藏着大量有用的信息,通过静态分析提取程序代码可能存在的漏洞点,用来构建测试用例以提高模糊测试的效率。实现一个原型系统,并在 4 家主流路由器厂商的 46 个路由器固件上进行测试,发现 16 个漏

收稿日期:2023-05-17

基金项目:国家自然科学基金面上项目(62072253)

通信作者:沙乐天(ltsha@njupt.edu.cn)

引用格式:王洪义,沙乐天.基于静态分析和模糊测试的路由器漏洞检测方法[J].信息安全研究,2024,10(1):40-47

洞,其中4个是零日漏洞.结果表明,与先进的自动化漏洞挖掘方法相比,该系统可以检测现有漏洞检测工具无法检测的漏洞.

关键词 静态分析;模糊测试;固件;漏洞挖掘;Web接口

中图法分类号 TP399

随着物联网的快速发展,越来越多的物联网设备可以在网络中访问,比如路由器、摄像头和打印机等.根据 Statista 的研究,到 2025 年,全球物联网设备的总装机量预计达到 309 亿台^[1].据统计,在所有物联网设备中,无线路由器和网络摄像头比其他嵌入式设备遭受的攻击更多,主要因为它们提供的 Web 服务暴露了可利用的漏洞.在 75% 的场景中,路由器充当物联网场景中的网关^[2],当路由器设备被攻击后局域网设备都存在被攻击的可能性.遗憾的是由于缺乏先进的防御机制和面向互联网的特性,路由器设备已成为黑客攻击的主要目标.

通常情况下大多数路由器都提供了基于 Web 的界面,用于管理和配置设备的功能.据统计,包含 Web 服务的无线路由器占其总数的 83.6%.常见的 Web 服务器是 HTTP 服务器,它通过 HTTP 协议提供基本的网络服务,后端二进制程序提供大量的接口,可以直接对设备进行操作.模糊测试是一种针对软件和系统十分有效的漏洞挖掘方法.该方法通过向被测设备发送大量非预期的输入,并通过监控其状态发现潜在的漏洞.本文认为,目前最先进的研究没有提取足够的固件信息来指导模糊测试,无效的测试用例导致模糊测试的低效和无效性.本文利用固件静态分析指导模糊测试中测试用例的构建,从而提高模糊测试的效率和准确性.本文主要贡献如下:

- 1) 提出一种新的基于共享键自动识别固件中多二进制漏洞的静态分析方法;
- 2) 提出一种新的基于静态分析与模糊测试相结合的漏洞检测方法,即通过静态分析生成的结果构建模糊测试的测试用例;
- 3) 设计并实现一个原型系统,可以检测现有 Fuzzer 无法触发的漏洞.该系统共发现 16 个漏洞,其中 4 个是零日漏洞.

1 相关工作

随着物联网问题的日益增多^[3],基于静态分析技术已经提出了不同的自动化漏洞检测工具^[4-6].Firmalice^[7]是一个二进制的分析框架,它利用符号执行和程序切片技术检测二进制固件中绕过身份验证的漏洞.李明琪等人^[8]通过权限校验分析和参数一致性分析方法,检测应用软件中存在的缺失对使用者的权限校验以及在发送控制指令时缺失保护字段的 2 类安全漏洞.DTaint^[9]通过使用指针别名分析以自下而上的方式构建敏感数据流,检测固件中的污点式漏洞.唐泉^[10]提出一种基于动态污点分析的输入数据字段分类方法,改进了基于代码覆盖率的遗传模糊测试方法.SaTC^[11]应用有针对性的数据流分析方法准确检测不可信用户输入的危险使用.不幸的是现有的静态分析工作一次只关注单个二进制程序或者模块,而忽略了后端多二进制之间不安全交互的检测.与动态分析相比,静态分析拥有更高的覆盖率,然而无法准确地确定代码执行的路径,这是静态分析的常见挑战.

动态分析特别是模糊测试,作为现在流行的漏洞检测技术,解决了静态分析无视实际运行环境以及代码约束的问题.SRFuzzer^[12]是一个用于测试物理路由器的全自动模糊框架,它首先从运行的设备捕获大量 Web 请求,然后对用户输入语义进行建模以生成测试用例.田里等人^[13]使用 Seq2Seq 模型进行二次学习,构建以协议管理、策略配置、详情查询模块为核心的物联网协议漏洞检测系统.此外,Boofuzz^[14]是一个先进的协议模糊器,需要手动编写协议格式描述文件以支持测试.PS-Fuzz^[15]是一个基于协议状态导向的灰盒模糊测试工具,它通过程序插桩以固件中与协议处理相关的字段为引导对目标设备进行模糊测试.不幸的是以上研究没有对目标设备的固件进行足够

的分析,无法生成有效的测试用例,从而受困于代码覆盖和攻击面,这是动态模糊分析的常见挑战。

2 研究背景与动机

本文主要研究针对路由器设备的漏洞检测,然而,现有的漏洞挖掘技术面临挑战:模糊测试阶段针对原始种子变异位置的确定以及静态分析阶段对于后端二进制之间不安全交互的检测。生成有效的测试用例至关重要,通过对数据进行有针对性的变异,可以提高测试用例的有效性,从而更好地揭示被测试程序中的潜在漏洞。本节利用真实设备中部分反编译代码来展示本文的发现。

本文发现路由器前端总是以键值对的形式发送请求数据,而后端则根据共享的键获取对应的值来解析该请求。图1所示为真实设备中的部分反编译程序,左侧显示出前端 GET 请求,右侧显示后端对该请求的处理程序。GET 数据包含 2 个参数键:action 与 usbName。在后端处理请求时,通过 sub_2BA8C 函数读取它们的值,并在没有进行任何过滤的情况下将其传递到 doSystemCmd() 函数中,从而造成命令注入漏洞。由此发现,前后端总是通过相同的关键词(如图1中的 action)进行数据通信,本文称其为共享键。本文认为共享键的识别是确定变异位置的关键,后端通过前端请求数据进行响应,恶意的输入未经过滤则造成漏洞。

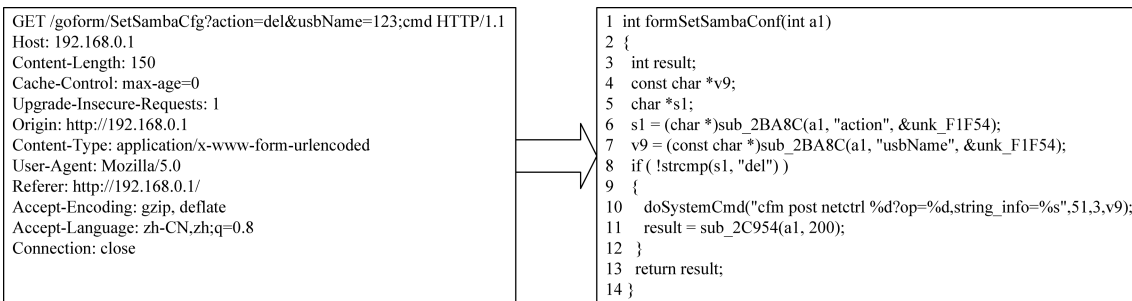


图1 前后端共享相同的键

现代物联网设备的固件是复杂的,据统计 86% 的固件是基于 Linux 的^[16],与其他基于 Linux 的系统类似,路由器设备中的固件包含大量相互依赖的二进制文件。在本文的漏洞检测过程中,这种交互是值得重视的。若只关注那些面向网络的二进制文件,就导致忽略与之关联的其他二进制文件中存在的漏洞。图2所示为真实设备中的部分反编译程序。在 mod_ssi.so 中(图2左端),第9行的 ssi_env_add() 函数将从请求中解析出的数据赋值

给 QUERY_STRING。在 downloadFile.cgi(图2右端)第6行中, getenv() 函数获取 QUERY_STRING 的值并将其赋值给变量 a。变量 a 在第8行被格式化后赋值给变量 b。最终,变量 b 的值被传入函数 system() 执行,从而出现漏洞。以往的针对嵌入式设备固件的静态分析缺乏对多二进制之间不安全交互的检测,本文认为可以通过识别具有共享键的相关函数(如表1所示)来建立后端多个二进制之间的依赖关系。然后依靠传统的数据流

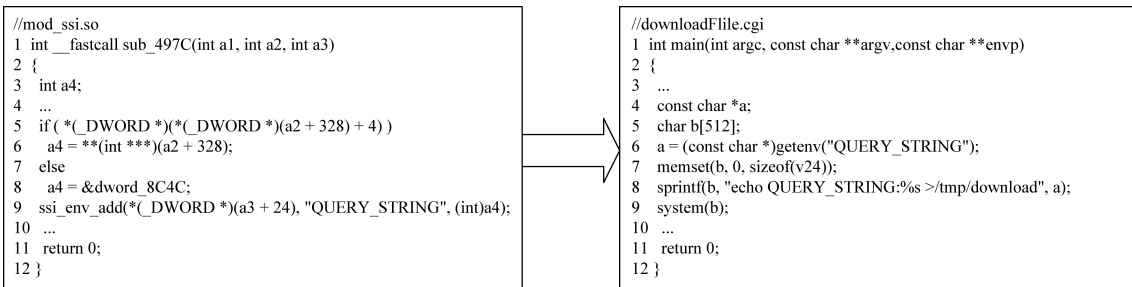


图2 多二进制通过共享键进行通信

表 1 常见的多二进制关联函数

类别	函数
存储类函数	<code>nvrnm_set()</code> , <code>nvrnm_safe_set()</code> , <code>setenv()</code> , <code>setValue()</code> , ...
读取类函数	<code>nvrnm_get()</code> , <code>nvrnm_safe_get()</code> , <code>getenv()</code> , <code>getValue()</code> , ...

分析实现多二进制不安全交互的检测。

3 模型设计

本文提出了一个针对路由器 Web 服务器的

模糊测试框架,用于检测缓冲区溢出漏洞与命令注入漏洞.该框架的漏洞检测模型如图 3 所示,原型系统主体由 3 大部分组成,分别为固件获取部分、静态分析部分和模糊测试部分,各个部分协同工作,完成路由器设备的漏洞检测工作。

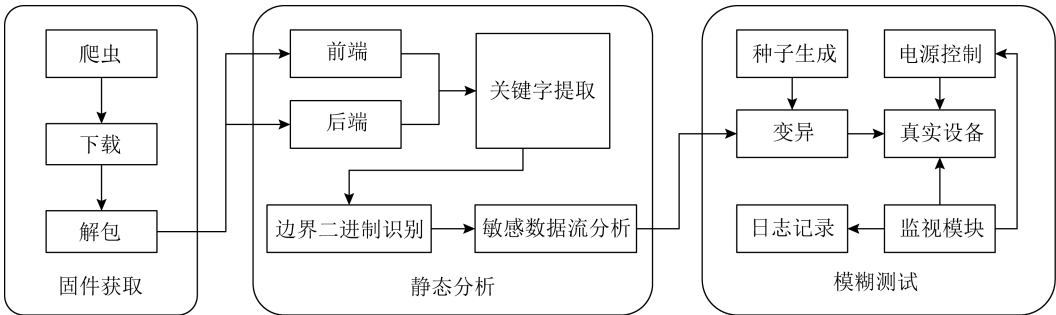


图 3 漏洞检测模型

3.1 固件获取

本部分主要包含 3 个子模块:爬虫模块、下载模块和解包模块.这些模块的主要功能是获取解包后的固件镜像,为后续的静态分析部分提供输入必要的数据基础。

首先,爬虫模块是负责从给定网站链接中爬取路由器固件镜像的 URL 的程序;其次,下载模块是用于下载固件文件的程序,该模块接收来自爬虫模块的 URL,并将其下载到指定的本地目录中;最后,解包模块是用于解包路由器固件镜像的程序。

3.2 静态分析

本部分主要包含 2 个子模块:关键字提取模块和敏感数据流分析模块.这些模块的主要功能是从固件中提取敏感信息,为后续的模糊测试样本构造提供依据。

3.2.1 关键字提取模块

本模块主要负责提取后端 Web 服务中与前端共享的关键字,为敏感数据流分析模块提供支持.如图 1 所示,前端请求的数据始终以键值对的形式出现.将在前端文件和后端二进制文件之间共享的关键字叫“共享键”.从前端提取这些共享键,并使用它们来定位处理前端请求的二进制程序

(下文称为“边界二进制”),然后将后端文件按照匹配数量降序排列,并选取匹配数量最高的 3 个文件作为边界二进制文件。

静态分析关键字提取过程见算法 1,该算法以固件镜像路径 `dir_path` 作为输入,最终得到二进制与其对应的关键字映射结构 `result`.函数 `list_files()` 用于根据传入类型返回目标文件,而 `extract_backend_keywords()` 与 `extract_backend_keywords()` 则负责进行关键字提取。

算法 1. 关键字提取算法.

输入:固件镜像存储路径 `dir_path`;
输出:固件二进制与其对应关键字的映射结构 `result`.

- ① FOR `frontend_file` in `list_files(dir_path, Type.frontend)`
- ② `frontend_keywords.update(extract_frontend_keywords(frontend_file))`;
- ③ END FOR
- ④ FOR `backend_file` in `list_files(dir_path, Type.backend)`
- ⑤ `backend_keywords=extract_backend_keywords(backend_file)`;


```

⑥ match_counts[backend_file]=count_
    matches(frontend_keywords,
        backend_keyword);
⑦ END FOR
⑧ sorted_matches=sort_by_value_
    desc(match_counts)[:3];
⑨ matched_keywords=frontend_keywords &
    backend_keywords;
⑩ result[backend_file]=matched_keywords;
⑪ END FOR
⑫ return result /* 返回匹配结果 */

```

3.2.2 敏感数据流分析模块

本模块主要负责根据关键字提取模块确定的边界二进制中的共享键为起点进行数据流分析。该模块直接检测共享键的引用位置,以此作为敏感数据流中的起点,从而避免从二进制的入口函数进行分析,它会检索所有由给定关键字出发并最

终到达危险函数的数据流路径,为模糊测试变异模块中变异位置的确定提供支持。准确检测固件中多个二进制文件之间的不安全交互是一项挑战。如表1所示,包含存储类函数的二进制被称为set类二进制,包含读取类函数的二进制文件被称为get类二进制。正如图2所示,多个二进制文件之间进行数据交换具有共同的特点,即存储函数和读取函数共享相同的关键字。因此,通过共享键建立2类文件之间的依赖关系。分析敏感变量的传递路径,收集所有可以达到表2中常见的危险函数的路径。

敏感路径生成过程见算法2。该算法以二进制与其对应的关键字映射结构为输入,最终得到敏感变量到危险函数的路径。`getCallChain()`用于获取与敏感关键字相关的数据流路径。`isContainsFocusFun()`函数判断敏感关键字的数据流路径中是否包含表2危险函数。

表2 常见的危险函数

类别	函数
命令注入类	<code>system()</code> , <code>execve()</code> , <code>shell_exec()</code> , <code>popen()</code> , <code>proc_open()</code> , ...
缓冲区溢出类	<code>strcpy()</code> , <code>strncpy()</code> , <code>sprintf()</code> , <code>snprintf()</code> , <code>strcat()</code> , <code>vsprintf()</code>

算法2. 敏感路径生成算法。

输入: 二进制文件与关键字映射路径 `bin_paray_mappings`;

输出: 敏感路径集合 `keywords_paths`。

```

① FOR bin_para_mapping in bin_paray_
    mappings
② cur_bin=read(bin_para_mapping.bin_
    path); /* 读取二进制 */
③ FOR keyword in bin_para_mapping.
    keywords
④ callChain=getCallChain(cur_bin,
    keyword);
⑤ flag=isContainsFocusFun(callChain);
    /* 进行危险函数检查 */
⑥ IF flag==true
⑦ keywords_paths.add(keyword,
    callChain);
⑧ END IF
⑨ END FOR
⑩ END FOR

```

```

⑪ return keywords_paths.
    /* 返回敏感路径集合 */

```

3.3 模糊测试

本节主要包含4个子模块:种子生成模块、变异模块、监视模块和电源控制模块。

3.3.1 种子生成模块

本模块主要负责生成模糊测试的初始种子。使用爬虫器和拦截器收集原始请求,并将其存储到本地文件中。爬虫器通过解析网页元素自动填充网页,特别是对于登录页面,使用预先编写的本地用户密码文件进行填充。而拦截器用于监视和拦截网络通信中的数据包。采用基于代理服务器的方法,捕获通过代理服务器的所有网络通信数据包,将其记录到本地文件中。

3.3.2 变异模块

本模块主要负责生成模糊测试的测试用例。首先标记变异位置,然后通过变异策略处理该位置的值。在变异位置标记中,主要借助静态分析部分敏感数据流分析模块的结果。只关注敏感变量(敏感数据流的起点)在原始数据包中的位置。如图4

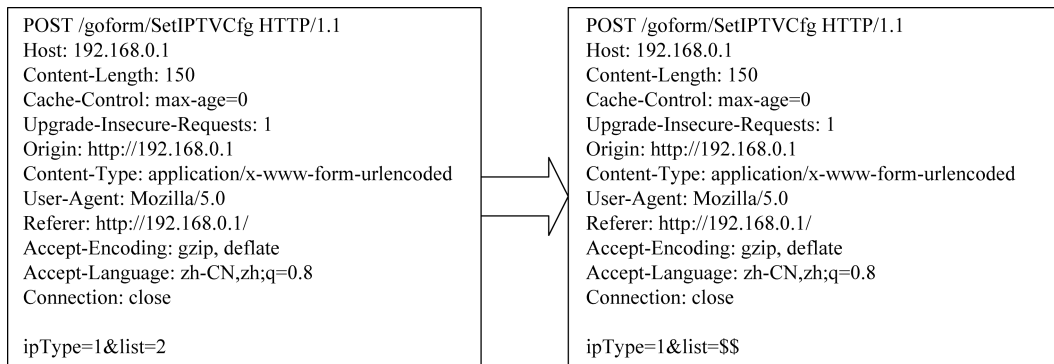


图 4 变异位置标记

所示,左侧为原始数据包,右侧为标记后的数据包。经静态分析,list 的值最终到达危险函数,用特殊字符 \$\$ 标记该位置。

根据不同的类型生成对应的参数值,主要采用以下变异策略:

- 1) 字符串.通过不断更改字符串的长度生成不同的随机字符串。
- 2) 数值.通过修改参数的边界值生成参数,以测试程序的边界情况。
- 3) 自定义命令.针对无响应的命令注入漏洞,基于 ping 等内置命令构造畸形有效载荷构造。

3.3.3 监视模块

本模块主要负责对目标设备状态的监视.监视模块包含 2 个监控器:基于响应的监控器和基于代理的监控器。

基于响应的监视器负责监测缓冲区溢出类漏洞以及有响应的命令注入类漏洞.满足以下条件之一时本模块会对异常情况进行日志记录操作。

- 1) 心跳监测.心跳监测机制,即设置定时程序对设备是否正常工作进行监控。

- 2) 响应差异分析.通过分析和对比正常请求和变异数据请求所得到的响应数据,实现对设备异常状态的检测。

基于代理的监视器负责监测无响应的命令注入类漏洞.变异模块定义了畸形的有效载荷来请求外部路由器.本文在本地网络中建立一个监视服务器用于接收路由器发送的请求.一旦命令注入被触发,本模块会对异常情况进行日志记录操作。

3.3.4 电源控制模块

本模块主要确保模糊测试的持续进行.该模块

由监视模块控制.如果后端服务陷入无响应状态,监视模块会向电源控制模块发送重启命令,以重新启动设备。

4 实验评估

4.1 数据集

本文选择 4 家提供固件在线下载服务的主流路由器设备厂商,包括 Tenda, D-Link, Netgear 与 TP-Link.如表 3 所示,选用 46 个固件样本,覆盖 ARM32 和 MIPS32 这 2 种架构.原型系统共处理了 377.6 M 的固件样本。

表 3 数据集样本

厂商	固件数量	固件大小/MB	架构
Tenda	14	104	ARM32/MIPS32
D-Link	15	152	MIPS32
Netgear	14	99.9	ARM32
TP-Link	3	21.7	MIPS32
总计	46	377.6	ARM32/MIPS32

4.2 固件获取的效率

如表 4 所示,固件获取部分从供应商的网站上爬取 531 个固件镜像的下载链接,并使用定制的下模块成功下载所有的固件镜像,由于部分固件存在加密,最终解包程序成功解包 435 个固件镜像,本模块下载成功率为 100%,解包成功率为 81.9%。出于本系统需要使用物理设备进行模糊测试,排除了个别价格昂贵的设备,选取了 46 个固件。

表 4 固件获取

步骤	固件镜像
爬虫	531
下载	531
解包	435
选取的固件样本	46

4.3 测试用例的有效性

为评估本系统构建测试用例的有效性,如表 5 所示,本文选取部分固件样本用来与先进的物联网设备模糊器 Boofuzz 和 SLoTFuzzer^[17] 进行比较.对于 Boofuzz,虽然可以在输入中插入特殊字符,但它对命令注入漏洞检测效果不佳;对于 SLoTFuzzer,由于在静态分析指导模糊测试阶段,只关注前端 Web 管理界面生成有状态的测试用例而忽略后端二进制对请求数据的限制,仅检测 3 个漏洞.

表 5 漏洞检测数据对比

漏洞编号	漏洞类型	Boofuzz	SLoTFuzzer	原型系统
CNVD-2021-95410	CI	×	×	√
CNVD-2021-95411	CI	×	×	√
CNVD-2021-103442	CI	×	×	√
CNVD-2022-03820	CI	×	×	√
CVE-2022-46538	CI	×	×	√
CVE-2014-3936	BoF	√	√	√
CVE-2013-7389	BoF	√	√	√
CVE-2015-2051	CI	×	√	√
CVE-2019-17621	CI	×	×	√

4.4 漏洞检测的成果

本文系统共发出 37 个异常警告,其中 24 个警告被认定为真实存在的漏洞,包括 12 个已知漏洞与 4 个零日漏洞(存在相同的异常警告属于同一漏洞的情况).我们已向国家信息安全漏洞共享平台报告,并被授予 CNVD 编号.漏洞检测成果如表 6 所示:

表 6 验证已知漏洞与发现未知漏洞

厂商	设备型号	漏洞类型	漏洞编号	未知/已知
Tenda	AC9/AC15/AC18	CI	CNVD-2021-95410	未知
		CI	CNVD-2021-95411	未知
	AC15/AC18	CI	CNVD-2021-103442	未知
		CI	CNVD-2022-03820	未知
	F1203	CI	CVE-2022-46538	已知
D-Link	DIR500	BoF	CVE-2014-3936	已知
	DIR645	BoF	CVE-2013-7389	已知
		CI	CVE-2015-2051	已知
	DIR859	CI	CVE-2019-17621	已知
	DIR866L	CI	CVE-2019-16920	已知
	DIR822	CI	CVE-2021-45608	已知
	DIR846	CI	CVE-2022-46641	已知
	Netgear R7000/R7000R	CI	CVE-2020-28373	已知
		BoF	CVE-2022-44199	已知
TP-Link	RE365	CI	CVE-2019-7406	已知
	TL-WR840N	CI	CVE-2022-25064	已知
总计	14	2	16	4/12

注:CI 表示命令注入漏洞,BoF 表示缓冲区溢出漏洞.

5 结 语

本文提出了一种新的基于静态分析和模糊测试相结合的路由器设备漏洞检测方法.该方法利

用静态分析生成的结果指导模糊测试阶段测试用例的构建,在静态分析阶段,通过识别具有键值特征的函数实现对多二进制之间的自动化漏洞检测;在模糊测试阶段,通过静态分析的结果构建出测试用例,使用更有针对性的突变策略来引发更

多漏洞.评估结果表明,本文方法优于目前最先进的
的方法,能够成功检测到现有 Fuzzer 无法触发的
漏洞.

参 考 文 献

- [1] Lionel S. Internet of things (IoT)—Statistics & facts [EB/OL]. [2023-01-31]. <https://www.statista.com/topics/2637/internet-of-things/>
- [2] Bojan J. Internet of things statistics for 2022—Taking things apart [EB/OL]. [2023-01-31]. <https://dataprot.net/statistics/iot-statistics>.
- [3] Alex H. IoT Security: 5 cyber-attacks caused by IoT security vulnerabilities [EB/OL]. [2023-01-31]. <https://www.cm-alliance.com/cybersecurity-blog/iot-security-5-cyber-attacks-caused-by-iot-security-vulnerabilities>
- [4] Qin F, Wang C, Li Z, et al. Lift: A low-overhead practical information flow tracking system for detecting security attacks [C] //Proc of the 39th Annual IEEE/PACM Int Symp on Microarchitecture (MICRO'06). Piscataway, NJ: IEEE, 2006: 135-148
- [5] Stephens N, Grosen J, Salls C, et al. Driller: Augmenting fuzzing through elective symbolic execution [C] //Proc of Network and Distributed System Security Symposium. Rosten, VA, USA: Internet Society, 2016: 1-16
- [6] Zhu D Y, Jung J, Song D, et al. TaintEraser: Protecting sensitive data leaks using application-level taint tracking [J]. ACM SIGOPS Operating Systems Review, 2011, 45 (1):142-154
- [7] Shoshitaishvili Y, Wang R, Hauser C, et al. Fomalice-automatic detection of authentication bypass vulnerabilities in binary firmware [C] //Proc of Network and Distributed System Security Symposium. Rosten, VA, USA: Internet Society, 2015: 1.1-8.1
- [8] 李明琪, 张磊, 杨哲慊. 面向智能路由器远程管理应用的漏洞检测工具[J]. 计算机应用与软件, 2020, 37(7): 266-274
- [9] Cheng K, Li Q, Wang L, et al. DTaint: Detecting the taint-style vulnerability in embedded device firmware [C] //Proc of the 48th Annual IEEE/PIFIP Int Conf on Dependable Systems and Networks (DSN). Piscataway, NJ: IEEE, 2018: 430-441
- [10] 唐泉. 基于动态污点分析的反馈式模糊测试改进方法[J]. 信息安全研究, 2019, 5(2): 145-151
- [11] Chen L, Wang Y, Cai Q, et al. Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems [C] //Proc of USENIX Security Symposium. Berkeley, CA: USENIX Association, 2021
- [12] Zhang Y, Huo W, Jian K, et al. SRFuzzer: An automatic fuzzing framework for physical SOHO router devices to discover multi-type vulnerabilities [C] //Proc of the 35th Annual Computer Security Applications Conf. New York: ACM, 2019: 544-556
- [13] 田里, 王晋. 面向工业物联网的协议漏洞检测技术研究 [J]. 信息技术与标准化, 2023 (5): 53-58
- [14] Pereyda J. Boofuzz [CP/OL]. [2023-01-31]. <https://github.com/jtpereyda/Boofuzz>
- [15] Li X, Pan X, Sun Y. PS-Fuzz: Efficient graybox firmware fuzzing based on protocol state [J]. Journal on Artificial Intelligence, 2021, 3(1): 21-31
- [16] Costin A, Zaddach J, Francillon A, et al. A large-scale analysis of the security of embedded firmwares [C] //Proc of the 23rd USENIX Security Symposium. Berkeley, CA: USENIX Association, 2014: 95-110
- [17] Zhang H, Lu K, Zhou X, et al. SIoTFuzzer: Fuzzing Web interface in IoT firmware via stateful message generation [J]. Applied Sciences, 2021, 11(7): 3120-3138



王洪义

硕士.主要研究方向为物联网安全、漏洞挖掘.

hongyi0228@163.com



沙乐天

博士,副教授.主要研究方向为软件安全、网络安全、物联网安全.

ltsha@njupt.edu.cn